

ЛЕКЦИЯ 3. Синхронизация процессов

Цель ОС – эффективное использование аппаратных и программных ресурсов вычислительной среды.

ТОГДА: ресурсы необходимо разделять между процессами, каждый момент времени аппаратные (физические) и программные (логические) ресурсы доступны только одному процессу.

КРИТИЧЕСКИЙ РЕСУРС – ресурс, допускающий обслуживание только 1 пользователя (процесса) в каждый момент времени. Процессы, использующие критический ресурс должны синхронизировать свои действия. Разделяемые переменные – пример критического ресурса.

КРИТИЧЕСКИЙ УЧАСТОК ПРОЦЕССА – участок, в котором происходит обращение к критическим ресурсам. Критические участки должны быть взаимно исключаемыми.

Механизм синхронизации процессов механизм недопущения одновременного выполнения критических участков – должен обладать 2-мя свойствами:

1. Если ≥ 1 процесса хотят войти в критический участок, то 1 из них должен получить разрешение войти в свой критический участок.
2. В каждый момент только 1 процессу разрешается находиться в своем критическом участке.

Синхронизация с помощью приемов нижнего уровня.

1. Блокировка памяти (операции над памятью – неделимы) = Алгоритм Деккера (для синхронизации 2-х процессов), трудно обобщается на произвольное число процессов.
2. Неделимая операция ПРОВЕРИТЬ и УСТАНОВИТЬ (ПиУ) = Test and Set Lock (TSL). Операция имеет два параметра ОБЩ (глобальная переменная разделяемая процессами), ЛОК (локальная переменная процесса) и выполняется в виде двух присвоений:

ПиУ(ЛОК,ОБЩ)==ЛОК:=ОБЩ;

ОБЩ:=1;

Значения переменной ОБЩ: 1 – блокирующее, 0 – разблокирующее.

begin int ОБЩ; ОБЩ:=0;

parbegin

ПРОЦЕСС1:

begin int ЛОК1; ЛОК1:=1;

do while (ЛОК1=1)

```

ПиУ(ЛОК1,ОБЩ);

end;

<критический участок>

ОБЩ:=0;

<оставшаяся часть процесса1>;

end;

```

ПРОЦЕСС2:

```

begin int ЛОК2; ЛОК2:=1;

do while (ЛОК2=1)

ПиУ(ЛОК2,ОБЩ);

end;

<критический участок>

ОБЩ:=0;

<оставшаяся часть процесса2>;

end;

parend; end;

```

Основной недостаток приемов нижнего уровня – **активное ожидание** входа в критический участок. На основе ПиУ можно построить МЬЮТЕКС – переменная, которая может находиться в 2-х состояниях – заблокированное и неблокированное. Инструмент, реализующий пассивное ожидание – семафоры.

СЕМАФОРЫ

Семафор – средство, позволяющее организовать пассивное ожидание (реализуется обычно в супервизоре).

Семафор **S** – целочисленная переменная, значения которой могут изменять неделимые (для прикладных процессов) операции **P(S)** и **V(S)**

Различают **семафоры общие** и **двоичные**

Операция **P(S)** – **попросить** ресурс, связанный с семафором **S**:

```

begin

S:=S-1;

if S>=0 then <процесс продолжает работать>

```

else <процесс останавливается и встает в очередь ожидания, связанную с семафором S , процесс остается заблокированным до тех пор пока операция $V(S)$, выполненная другим процессом не разблокирует его>;

end;

Операция $V(S)$ – вернуть ресурс, связанный с семафором S :

begin

$S:=S+1$;

if $S>0$ then <процесс продолжает работать>

else <один процесс удаляется из очереди ожидания, получает ресурс и разрешение продолжить работу; продолжает работать либо процесс, который обратился в операции $V(S)$, либо процесс, получивший ресурс>;

end;

Область значений общего семафора S :

$-Pr \leq S \leq Res$,

Pr = количество конкурирующих процессов,

Res = количество единиц ресурса.

При $S=-$ количество конкурирующих процессов: имеем ТУПИК!

Область значений двоичного семафора S : $-1 \leq S \leq +1$

$S=+1$ – ни один из процессов не находится в своем критическом участке,

$S=0$ – один из процессов находится в своем критическом участке,

$S=-1$ – один из процессов находится в своем критическом участке и один процесс – в очереди в семафоре S .

Задача **ПОСТАВЩИК – ПОТРЕБИТЕЛЬ** решается с помощью семафоров

begin int БУФ, \$общий семафор – текущее количество свободных буферов\$

ЗАНЯТО, \$общий семафор – текущее количество занятых буферов\$

СИНХРО; \$двоичный семафор – гарантирует, что в каждый момент времени только 1 процесс работает с буферными стрелками \$

БУФ:=<исходное количество свободных буферов в системе>;

ЗАНЯТО:=0<исходное количество занятых буферов в системе>;

parbegin

ПОСТАВЩИК:

begin

do while (true)

<приготовить сообщение>

P(БУФ); P(СИНХРО);

<послать сообщение>

V(ЗАНЯТО); V(СИНХРО);

end; end;

ПОТРЕБИТЕЛЬ:

begin

do while (true)

P(ЗАНЯТО); P(СИНХРО);

<получить сообщение>;

V(БУФ); V(СИНХРО);

<обработать сообщение>;

end; end;

parend; end;

Семафоры не всегда удобны и приводят к сложным не прозрачным конструкциям в случае необходимости организации совместной работы с множеством типов ресурсов и обработкой набора событий.

Монитор управления взаимодействием параллельных процессов:

1. Разделяемая управляющая структура данных (централизация управляющей информации)
2. Набор функций доступа к структуре данных для управления синхронизацией и коммуникациями между процессами (функции – неделимые операции для приложений=реализованы в ядре).

+: 1. Избавляет программиста от деталей реализации работы с разделяемыми данными;

2. Используется для организации сложных ожиданий (совокупности ресурсов и событий, одного из ресурсов или события из некоторого множества).

Пример реализации: **Монитор на основе таблицы синхронизации (ТС)**

Принципы:

1. Синхронизация взаимодействующих процессов осуществляется через анализ и последующее изменение полей **ТС**.
2. В **ТС** представлены все синхронизируемые процессы, упорядоченные по убыванию приоритета.
3. Наименьший приоритет имеет вспомогательный процесс, постоянно находящийся в состоянии готовности. Получает управление при блокировании всех взаимодействующих процессов (ТУПИКе) и устраняет тупиковую ситуацию.
4. В строке, соответствующей каждому процессу содержится: а) адрес области сохранения; б) переменная состояния процесса (готов/не готов); в) тайм-аут ожидания выделения ресурсов или наступления событий; г) ссылка на список ресурсов, один из которых ожидает процесс; д) события, наступление которых может привести к переключению процессов.

Процесс	Область сохр	Готовность	Таймер	Ресурсы	Событие1	Событие2	...	СобытиеN
P1	Обл1	Не готов	0	Список1	0	0		0
P2	Обл2	Не готов	5	Список2	0	1		0
P3	Обл3	Готов	0	0		0		0
P4	Обл4	Не готов	9	0	1	1		1
P5	Обл5	Готов	0	0	0	0		0
P6	Обл6	Готов	0	0	0	0	0	0

Функции управления и синхронизации процессов реализуются с помощью процедур монитора: **TP**, **TV**, **POST**, **WAIT**, которые анализируют и изменяют поля **ТС**. Номер строки текущего процесса присваивается переменной монитора **ТЕКУЩИЙ**. Текущим может стать только самый приоритетный процесс из всех готовых к работе.

Логика операций монитора.

Операцией **TP** процесс пользуется при необходимости получения критических ресурсов, локализованных внутри тела монитора. Обращаясь к монитору процесс, может указать через параметры: а) несколько семафоров с различными видами ресурсов, одним из которых процесс хотел бы обладать; б) набор событий, наступления одного из которых ожидает процесс; в) тайм-аут ожидания одного из ресурсов или событий. Процесс задерживается на время поиска запрошенных ресурсов. При наличии хотя бы одного запрошенного вида ресурса (идентифицируемого семафором) получает в область сохранения ответ, содержащий вид выделенного ресурса и его адрес, активизируется и продолжает работу. В противном случае согласно запросу заполняется строка **ТС**, отведенная этому процессу. Обратившийся к операции запроса процесс блокируется, а запускается наиболее приоритетный готовый процесс.

TP(указатель на список семафоров ресурсов, указатель на список событий, тайм-аут)

begin

<задержать текущий процесс>;

for i:=1 to <число семафоров в запросе> do

if <счетчик i-го семафора >0> then

<взять ресурс из очереди; передать в область сохранения процесса адреса семафора и ресурса; активизировать задержанный процесс>;

end; end;

ТАБЛИЦА[ТЕКУЩИЙ, ГОТОВНОСТЬ]:= НЕ ГОТОВ;

ТАБЛИЦА[ТЕКУЩИЙ, ТАЙМЕР]:= ТАЙМ-АУТ;

ТАБЛИЦА[ТЕКУЩИЙ, РЕСУРСЫ]:= АДРЕС СПИСКА СЕМАФОРОВ;

<отметить в соответствующих столбцах таблицы заказанные в запросе события>;

<активизировать наиболее приоритетный готовый процесс>;

end;

Операцией **TV** процесс возвращает монитору освободившийся ресурс. Поиск процесса, ожидающего этот ресурс, осуществляется сканированием столбца **ТС РЕСУРСЫ** и просмотром списков ожидаемых каждым процессом ресурсов. Из всех ожидающих этот ресурс процессов находится самый приоритетный, в область сохранения которого передаются адреса семафора и ресурса. В зависимости от приоритета активизируется либо задержанный процесс, либо разблокированный. При отсутствии процессов, ожидающих этот ресурс, он возвращается в очередь к своему семафору и активизируется задержанный процесс.

TV(семафор, указатель на ресурс)

begin

<задержать текущий процесс>;

for i:=1 to <число строк в таблице> do

if ТАБЛИЦА[ТЕКУЩИЙ, РЕСУРСЫ]_{не}=0 then

if <список ожидаемых ресурсов содержит запрос на возвращаемый ресурс> then

<передать в область сохранения процесса P_i адрес семафора и ресурса>;

for j:=3 to <число столбцов в таблице> do

ТАБЛИЦА[i, j]:= 0;

end;

ТАБЛИЦА[i, ГОТОВНОСТЬ]:=ГОТОВ;

if i<ТЕКУЩИЙ then <активизировать P_i> else <активизировать задержанный процесс>

end; end; end; end;

<возвратить ресурс в очередь к семафору>;

<активизировать задержанный процесс>;

end;

Операцией **WAIT** процесс может заблокировать себя до наступления одного из нескольких событий. Можно задать тайм-аут ожидания событий. В столбцах **ТС** отмечаются перечисленные в запросе события.

WAIT(список событий, тайм-аут)

begin

<задержать текущий процесс>;

ТАБЛИЦА[ТЕКУЩИЙ, ГОТОВНОСТЬ]:= НЕ ГОТОВ;

ТАБЛИЦА[ТЕКУЩИЙ, ТАЙМЕР]:= ТАЙМ-АУТ;

<отметить в соответствующих столбцах таблицы заказанные в запросе события>;

<активизировать наиболее приоритетный готовый процесс>;

end;

Операцией **POST** процесс может сигнализировать о наступлении события. В результате наиболее приоритетный процесс из числа ожидающих данное событие переводится в состояние готовности, но активизируется только в том случае, если его приоритет выше приоритета процесса, выдавшего операцию **POST**.

POST(событие)

begin

<задержать текущий процесс>;

for i:=1 to <число строк в таблице> do

if ТАБЛИЦА[i, СОБЫТИЕ]≠0 then

for j:=3 to <число столбцов в таблице> do

ТАБЛИЦА[i, j]:=0;

end;

<переслать в область сохранения информацию о совершившемся событии>;

ТАБЛИЦА[i, ГОТОВНОСТЬ]:= ГОТОВ;

if i<ТЕКУЩИЙ then <активизировать P_i> else <активизировать задержанный процесс>

end; end; end;

<активизировать задержанный процесс>;

end;

Для обеспечения временной синхронизации необходим процесс **CLOCK**, имеющий бесконечный цикл работы и приоритет, равный приоритету процедур монитора. Процесс использует системный таймер вычислителя и запускает на один цикл работы таймера текущий процесс, после чего процесс **CLOCK** уменьшает на единичный интервал ненулевые элементы столбца ТАЙМЕР ТС, переводит процессы с истекшими тайм-аутами в состояние готовности, выбирает наиболее приоритетный готовый процесс и изменяет согласно выбору переменную ТЕКУЩИЙ.

Алгоритм процесса CLOCK

begin

do while (true) <запустить на 1 цикл работы таймера ТЕКУЩИЙ процесс>

<задержать текущий процесс>;

for i:=1 to <число строк в таблице> do

if ТАБЛИЦА[i, ТАЙМЕР]≠0 then

ТАБЛИЦА[i, ТАЙМЕР]:= ТАБЛИЦА[i, ТАЙМЕР]-1;

if ТАБЛИЦА[i, ТАЙМЕР]=0 then

for j:=3 to <число столбцов в таблице> do

ТАБЛИЦА[i, j]:=0;

end;

<переслать в область сохранения информацию о истекшем тайм-ауте>;

ТАБЛИЦА[i, ГОТОВНОСТЬ]:= ГОТОВ;

end; end; end; end;

Описанный монитор позволяет реализовать сложную схему меж процессного взаимодействия и решить задачу параллельного ожидания ресурсов и событий. Управление взаимодействием процессов с помощью ТС имеет ясную логическую структуру, является наглядным и простым (обусловлено концентрацией управляющей информации в ТС). Модификация меж процессных связей и появление новых процессов приведет к незначительным изменениям в ТС без изменения логики процедур.

ТУПИКИ

Ситуация, когда процессы ждут друг друга неопределенно много – ТУПИК!

Необходимые (но не достаточные) условия тупика:

1. Условие **взаимоисключения** (процессы требуют монопольного управления выделенными им ресурсами).
2. Условие **удержания и ожидания** ресурсов (процессы удерживают за собой уже выделенные ресурсы и ожидают выделения дополнительных ресурсов).
3. Условие **неперераспределяемости** ресурсов (ресурсы нельзя отобрать у процессов, удерживающих их, пока они не будут использованы для завершения работы).
4. Условие **кругового ожидания** (существует кольцевая цепь процессов, в которой каждый процесс удерживает за собой один или более ресурсов, требующихся следующему процессу цепи).